

Day 40: Write Transactions to Text File

Day 40 — Write Transactions to Text File

1. Day Number

Day 40

2. Topic Name

Basic File Writing

Today you will learn how Python can save information permanently inside a **text file**.

Main topics:

- open()
 - write mode "w"
 - with
 - write()
 - writing one item per line
-

3. Connection

Previously, you created and worked with a **transaction history stored in a list**.

For example:

```
transactions = [  
    "Bought Laptop - 50000",  
    "Bought Mouse - 1200",  
    "Bought Keyboard - 2500"  
]
```

The problem is that a Python list normally exists only while the program is running.

When the program stops, that data disappears.

Today you will learn how to save the transaction history into a file.

```
Python List  
  |  
  v  
Open File  
  |  
  v  
Write Transactions  
  |  
  v  
transactions.txt
```

This is your first step toward **persistent data storage**.

4. Important Topics

Today focus on these concepts:

open()

Used to open or create a file.

```
open("example.txt", "w")
```

Write Mode "w"

"w" means:

```
write mode
```

It allows Python to write data into the file.

Example:

```
open("notes.txt", "w")
```

Important:

| If the file already contains data, "w" mode replaces the old content.

with

The recommended way to work with files is:

```
with open(...) as file:
```

Python automatically closes the file when the block finishes.

write()

Used to write text into the file.

Example:

```
file.write("Hello")
```

5. Foundational Notes

What is a file?

A file stores information on your computer.

Examples:

```
transactions.txt  
products.txt
```

```
notes.txt
report.txt
```

Unlike a normal Python variable, file data can remain after your program finishes.

Memory vs File

A list exists in program memory:

```
transactions = [...]
```

Conceptually:

```
Program Running
  |
  v
Memory
  |
  +----- transactions list
```

When the program ends:

```
Memory data disappears
```

But if we save the data:

```
Program
  |
  v
transactions.txt
  |
  v
Saved on disk
```

The data can be used later.

Basic open() syntax

The general pattern is:

```
open("filename", "mode")
```

For example:

```
open("transactions.txt", "w")
```

Here:

```
transactions.txt
```

is the filename.

And:

```
"w"
```

means write mode.

Why use with?

You could technically open a file manually, but beginners should prefer:

```
with open("file.txt", "w") as file:  
    ...
```

The major advantage is that Python handles closing the file automatically.

Conceptually:

```
with block starts  
    |  
    v  
Open file  
    |  
    v  
Use file  
    |  
    v  
with block ends  
    |  
    v  
File automatically closed
```

Understanding write()

Suppose:

```
text = "Payment received"
```

You can write it using:

```
file.write(text)
```

But there is an important detail.

`write()` does **not automatically move to the next line**.

For example:

```
file.write("Apple")  
file.write("Banana")
```

could produce:

```
AppleBanana
```

instead of:

```
Apple  
Banana
```

The newline character `\n`

Use:

```
"\n"
```

to create a new line.

Example:

```
file.write("Apple\n")  
file.write("Banana\n")
```

The file becomes:

```
Apple  
Banana
```

For today's problem, this is very important because you want:

one transaction per line.

6. Easy Example

Consider some names:

```
names = ["Amit", "Rahul", "Priya"]
```

We want a file that looks like:

```
Amit  
Rahul  
Priya
```

The basic idea is:

```
with open("names.txt", "w") as file:  
    for name in names:  
        file.write(name + "\n")
```

What is happening?

```
names list  
|
```

```
    v
  for loop
    |
    v
  take one name
    |
    v
  add "\n"
    |
    v
  write to file
```

Iteration 1:

```
"Amit" + "\n"
```

Iteration 2:

```
"Rahul" + "\n"
```

Iteration 3:

```
"Priya" + "\n"
```

Result:

```
names.txt

Amit
Rahul
Priya
```

This demonstrates the technique you need today without solving the transaction problem for you.

7. Problem Statement

Create a list containing transaction strings.

For example, your transactions might represent:

```
Payment received
Product purchased
Refund completed
```

Your program should:

1. Create a list of transaction strings.
2. Open a text file in write mode.
3. Loop through the transaction list.
4. Write each transaction into the file.
5. Put each transaction on a separate line.

Conceptually:

```
transactions list
  |
  v
Open transactions.txt
  |
  v
Loop through transactions
  |
  v
Write transaction + newline
  |
  v
transactions.txt
```

Do not worry about reading the file yet. Today's focus is only **writing**.

8. Concepts Used

You will use:

List

Stores multiple transactions.

```
transactions = [...]
```

with

Manages the file safely.

```
with ...
```

open()

Opens or creates the text file.

```
open(...)
```

Write mode

```
"w"
```

allows writing into the file.

for loop

Processes transactions one by one.

Conceptually:

```
for each transaction
    |
    v
write transaction
```

write()

Writes text into the file.

```
file.write(...)
```

Newline

```
"\n"
```

puts the next transaction on the next line.

9. Thought Process

Before writing Python code, think about the problem in small steps.

Step 1: What data do I have?

You have multiple transactions.

Therefore, a list is suitable.

```
transactions
```

contains strings.

Step 2: Where should the transactions be saved?

Inside a text file.

For example:

```
transactions.txt
```

Step 3: Which file mode do I need?

You want to write data.

Therefore:

```
"w"
```


Step 4: How do I process every transaction?

Use a loop.

```
Transaction 1
Transaction 2
Transaction 3
...
```

Step 5: How do I keep transactions on separate lines?

Add:

```
"\n"
```

after each transaction.

Complete mental model

```
START
|
v
Create transaction list
|
v
Open file in "w" mode
|
v
Take one transaction
|
v
Write transaction + "\n"
|
v
More transactions?
|      \
Yes      No
|      \
+-----v
        Close file
        |
        v
        END
```

With with, Python handles the closing part automatically.

10. Pseudocode

```
START

Create a list of transaction strings
```

```
Open a text file using write mode

    FOR each transaction in transaction list

        Write transaction to file
        Write a newline after transaction

    END FOR

File closes automatically

END
```

Notice that pseudocode describes the **logic**, not exact Python syntax.

11. Suggested Solving Approach — Functional Approach

Since you are practicing the functional approach, separate the file-writing responsibility into a function.

Think about a function such as:

```
save_transactions(...)
```

Its responsibility should be:

```
Receive transactions
    |
    v
Open file
    |
    v
Write transactions
```

Conceptually:

```
Main Program
    |
    | transactions
    v
save_transactions()
    |
    v
transactions.txt
```

A possible design could be:

```
FUNCTION save_transactions(transaction_list)

    open file

    FOR every transaction
```

```
        write transaction
    END FOR

END FUNCTION
```

Then the main part of your program only needs to:

```
Create transactions
    |
    v
Call save function
```

This is better than putting every operation directly into one large block because the function has one clear responsibility:

Save transactions to a file.

12. Easy Edge Cases

Edge Case 1: Empty Transaction List

Suppose:

```
transactions = []
```

There is nothing to write.

Your loop simply runs zero times.

The resulting file may be empty:

```
transactions.txt
```

```
<empty>
```

That is acceptable for today's exercise.

Edge Case 2: Existing File

Suppose transactions.txt already contains:

```
Old Transaction 1
Old Transaction 2
```

Opening it using:

```
"w"
```

will overwrite the existing content.

After writing new transactions, the old transactions will no longer be there.

Remember:

```
"w"
 |
 v
Write new content
 +
Replace existing content
```

Later you can learn about append mode:

```
"a"
```

but it is not required today.

13. Expected Input

For today's exercise, you do **not necessarily need** `input()` from the keyboard.

Your input can simply be a Python list containing transaction strings.

Conceptually:

```
[
    transaction 1,
    transaction 2,
    transaction 3
]
```

Example data:

```
Payment received - 1000
Bought keyboard - 2500
Refund - 500
```

The important thing is that every list item should be a string.

14. Expected Output

Suppose your transaction list conceptually contains:

```
Payment received - 1000
Bought keyboard - 2500
Refund - 500
```

Your program should create:

```
transactions.txt
```

The file should contain:

```
Payment received - 1000
Bought keyboard - 2500
```

Refund - 500

Notice:

```
Transaction 1
\n
Transaction 2
\n
Transaction 3
```

Each transaction appears on its **own line**.

15. Hint Only

Think about these four pieces:

```
with open("transactions.txt", "w") as file:
```

Then ask yourself:

```
How can I visit every transaction?
```

You already know the answer from earlier days:

```
for ...
```

Inside that loop, think about:

```
file.write(...)
```

Finally, remember that `write()` does not automatically create a new line.

You will probably need:

```
"\n"
```

So your overall structure should look mentally like:

```
FUNCTION
  |
  v
open file
  |
  v
for each transaction
  |
  v
write transaction + ???
```

Your task is to figure out what replaces `???` and put the pieces together yourself.